

PLATAFORMA DE TELEMEDICINA · MÓDULO 4 · VERSÃO 1.1

Segurança por Linha (RLS) e Primeiras Rotas

O banco passa a decidir sozinho quem vê cada linha; a API ganha o porteiro (login), o primeiro fluxo de negócio (cadastro de perfil) com auditoria na mesma transação, e um roteiro que prova tudo isso de ponta a ponta contra o Supabase — criando e apagando o próprio usuário de teste.

O que este módulo pressupõe

O Módulo 3 terminou: `pnpm verificar` dizia “Tudo certo” com o passo [6/6], `localhost:3000` mostrava quatro luzes verdes, e o commit do módulo foi enviado ao GitHub.

O projeto que acompanha este PDF é `telemedicina-modulo-04.zip`. Contém tudo que o texto cita, no caminho citado, e foi testado do zero: instalação, tipos nos 4 pacotes, 13 testes, as 3 migrações num Postgres limpo com o schema `auth` e os papéis `anon/authenticated` do Supabase, as políticas exercitadas como paciente, médico, admin e anônimo, as rotas chamadas com token válido, inválido e ausente, API e site no ar com cinco luzes verdes.

0. Onde você vai chegar

Luz	O que ela prova
Site, API, Supabase, Banco	As quatro do Módulo 3, iguais.
Segurança por linha (RLS) — nova	RLS ligado nas 5 tabelas, as 14 políticas da migração 0002 presentes, a função <code>papel_atual()</code> existe e ninguém consegue alterar ou apagar a auditoria.

É um comando novo que vale mais que a luz: `pnpm api:testar-fluxo`. Ele cria um usuário no Supabase Auth, faz login como ele com a chave pública (exatamente como o site fará), chama as rotas da API com o token, tenta furar o RLS por fora da API, e apaga o usuário. Termina com “Tudo certo” ou diz em qual passo falhou.

Novo	O que é
<code>packages/db/drizzle/0002_rls_e_politicas.sql</code>	Migração manual: liga o RLS, cria <code>papel_atual()</code> e 14 políticas, revoga UPDATE/DELETE da auditoria.
<code>POST /perfis</code> , <code>GET /perfis/eu</code> , <code>GET /medicos</code>	As primeiras rotas de negócio. Exigem login.
<code>pnpm db:verificar-rls</code>	Confere o RLS (também é o passo [7/7] do <code>pnpm verificar</code>).
<code>pnpm api:testar-fluxo</code>	Prova de ponta a ponta, contra o Supabase real.

1. Os conceitos

Até aqui, quem tem a chave pública do Supabase (e ela está no navegador de todo mundo — Módulo 2) podia, em tese, fazer `select * from pacientes` direto no banco. Nada impedia. Este módulo fecha essa porta.

Palavra	O que é (versão de 10 anos)	No nosso projeto
RLS (Row Level Security)	Um segurança em cada tabela que olha quem está pedindo e mostra só as linhas que essa pessoa pode ver. Ligado sem regras, ele não mostra nada para ninguém.	Ligado nas 5 tabelas pela migração 0002.
Política (policy)	Uma regra do segurança: “para SELECT, deixe passar a linha se ela for da própria pessoa”. Cada tabela tem várias, uma por operação e situação.	14 políticas, todas em <code>0002_rls_e_politicas.sql</code> .
<code>auth.uid()</code>	A pergunta “quem é você?” que o Supabase responde lendo o token do login. Sem login, responde vazio.	Usada em quase toda política.
USING / WITH CHECK	USING: quais linhas a pessoa alcança (ler, alterar, apagar). WITH CHECK: o que ela pode gravar (inserir, ou o novo valor de uma alteração).	É o que impede alguém de se promover a admin.

Palavra	O que é (versão de 10 anos)	No nosso projeto
Token (JWT)	O crachá que o Supabase entrega no login. O site manda esse crachá em cada chamada; a API confere com o Supabase de quem ele é.	Authorization: Bearer
Chave secreta ignora o RLS	A chave da API passa por cima de qualquer política. Por isso ela nunca sai da API — e por isso é a API quem grava, sempre com auditoria.	ADR-0003 + ADR-0006.

As três camadas que se cobrem (ADR-0006)

- 1. A API escreve.** Cadastro, agendamento, tudo que muda dado passa pela API, que valida, grava e registra auditoria na mesma transação.
- 2. O banco decide quem lê.** Mesmo que o site tenha um bug, mesmo que alguém use a chave pública fora do site, o RLS garante que cada um vê só o que é seu.
- 3. A auditoria não se apaga.** Não há política de UPDATE nem DELETE para usuários, e ainda revogamos o privilégio. Duas travas para a mesma coisa.

2. O que há de novo no projeto

```
telemedicina/
|-- packages/db/
|   |-- drizzle/0002_rls_e_politicas.sql    liga o RLS, papel_atual(), 14 politicas, REVOKE
|   |-- src/estado-banco.ts                + inspeciona RLS, politicas, privilegios da auditoria
|   |-- src/verificar-rls.ts                pnpm db:verificar-rls (e o [7/7] do verificar)
|-- apps/api/src/
|   |-- autenticacao.ts                    o porteiro: le o token, pergunta ao Supabase quem e, ou 401
|   |-- auditoria.ts                       registrarAuditoria(): uma linha por acao, na mesma transacao
|   |-- erros.ts                           toda falha vira { ok:false, erro:{codigo, mensagem, detalhes?} }
|   |-- rotas/perfis.ts                     POST /perfis e GET /perfis/eu
|   |-- rotas/medicos.ts                   GET /medicos
|   |-- rotas.test.ts                      5 testes das rotas, sem banco e sem rede
|   |-- servidor.ts                        registra tudo acima; /saude ganha "seguranca"
|   |-- testar-fluxo.ts                    pnpm api:testar-fluxo
|-- apps/web/src/app/page.tsx              quinta luz: Seguranca por linha (RLS)
|-- packages/shared/src/index.ts           StatusSaude.seguranca, PerfilResposta, MedicoResumo, Papel
|-- scripts/verificar-ambiente.mjs         passo [7/7]
`-- docs/adr-0006-rls-como-ultima-defesa.md
```

Dependências novas na API: @supabase/supabase-js (para perguntar ao Supabase Auth de quem é o token e, no roteiro de teste, criar e apagar usuários) e drizzle-orm (as rotas consultam o banco com tipos). Nenhum arquivo de segredo novo: a API continua lendo só apps/api/.env.

3. As políticas, tabela por tabela

Leia packages/db/drizzle/0002_rls_e_politicas.sql com esta tabela ao lado. Cada linha aqui é uma CREATE POLICY lá. “Próprio” significa id = auth.uid() (ou perfil_id = auth.uid()).

Tabela	Quem vê (SELECT)	Quem insere	Quem altera	Quem apaga
perfis	O próprio; admin vê todos	Ninguém pelo banco — só a API	O próprio, sem mudar o papel	Ninguém
medicos	Qualquer pessoa logada (é a lista para escolher com quem marcar)	Só a API	O próprio médico	Ninguém
pacientes	O próprio; o médico que tem consulta com ele; admin	Só a API	O próprio paciente	Ninguém
consultas	O paciente e o médico da consulta; admin	O paciente, marcando a própria	Os dois lados	Ninguém
auditoria	Só admin	Qualquer logado, em próprio nome (quem = auth.uid())	Ninguém (+ REVOKE)	Ninguém (+ REVOKE)

3.1 Por que existe papel_atual()

A política “admin vê todos” precisa saber o papel de quem pergunta — e o papel está em perfis. Mas uma política de perfis que consulta perfis dispara a própria política de novo, e de novo, sem fim (o Postgres devolve “infinite recursion detected”). A saída padrão do Supabase é uma função SECURITY DEFINER: ela roda como o dono do banco, ignorando o RLS, e devolve só o papel. Duas proteções nela: STABLE (o Postgres pode chamá-la uma vez por consulta, não por linha) e SET search_path = '' (ninguém consegue trocar a tabela perfis por outra de mesmo nome em outro schema).

3.2 Por que ninguém muda o próprio papel

A política de UPDATE em perfis tem WITH CHECK (... AND papel = papel_atual()): o novo valor de papel tem de ser igual ao atual. Um paciente pode trocar o telefone, mas se tentar update perfis set papel = 'admin' o banco responde “new row violates row-level security policy”. Promover alguém é tarefa da API, com auditoria, em módulo futuro.

3.3 O detalhe (SELECT auth.uid())

Nas políticas, `auth.uid()` aparece dentro de um `(SELECT ...)`. É a forma recomendada pelo Supabase: o Postgres avalia uma vez por consulta em vez de uma vez por linha. Em tabela de milhões de linhas, é a diferença entre milissegundos e segundos.

4. A API: porteiro, rotas e auditoria

4.1 O porteiro (`autenticacao.ts`)

Toda rota protegida declara `preHandler: exigirLogin`. O porteiro lê `Authorization: Bearer`, pergunta ao Supabase Auth (com a chave secreta) de quem é o token, e ou guarda o usuário em `req.usuario` ou responde 401 no envelope padrão — a rota nem chega a rodar. Preferimos perguntar ao Supabase a validar o token localmente: um usuário bloqueado ou apagado deixa de entrar na hora, sem esperar o token expirar.

O porteiro recebe o “autenticador” de fora (`criarServidor({ autenticar })`). É isso que permite os testes usarem um autenticador falso e provarem 401 e 400 sem rede.

4.2 As rotas

Rota	Corpo / resposta	Regras
POST /perfis	Corpo: papel (paciente ou medico), nomeCompleto, telefone?, medico {crm, crmUf, especialidade?} ou paciente {dataNascimento, cpf?}. Resposta 201 com o perfil completo.	Valida com Zod (400 com a lista de campos). papel admin não é aceito. Grava perfis + medicos/pacientes + auditoria numa transação. Repetido: 409.
GET /perfis/eu	Resposta 200 com id, papel, nome, telefone, criadoEm e o bloco medico ou paciente.	404 se a pessoa logada ainda não tem perfil.
GET /medicos	Lista de {id, nomeCompleto, crm, crmUf, especialidade}, em ordem de nome.	Não é dado clínico: não gera auditoria.

4.3 Auditoria na mesma transação

Em rotas/perfis.ts, o `banco.transaction(async (tx) => {...})` envolve os três INSERTs e o `registrarAuditoria(tx, ...)`. Se qualquer um falhar, nenhum fica. Não existe perfil sem a linha “perfil.criado” na auditoria, nem o contrário. O registro guarda quem, a ação, a tabela, o id e o IP — e em detalhes só `{ papel }`, nunca dado clínico.

4.4 Erros em um lugar só (`erros.ts`)

Erro do Zod vira 400 com detalhes: `[{campo, problema}]`. `ErroHttp(404, 409...)` vira o status e o código pedidos. Erro do próprio Fastify (JSON malformatado, rate limit) mantém o status dele. Qualquer outra coisa é 500 genérico para o cliente e detalhado no log — o usuário nunca vê stack trace, e o token nunca vai para o log (redact).

5. Atualizar o projeto e aplicar a migração

Ordem obrigatória (errata da v1.0)

1) Ctrl+C no `pnpm dev`. 2) Extrair o zip por cima. 3) `pnpm install` antes de qualquer outro comando. A API ganhou dependências novas; sem o `install` ela não sobe e as cinco luzes ficam vermelhas com “Cannot find package”.

1. Pare o `pnpm dev` (Ctrl+C). Extraia o zip por cima da pasta do projeto, substituindo (mesmo procedimento do Módulo 3; os `.env` não são tocados).

2. Na raiz:

```
pnpm install
pnpm typecheck
pnpm test
```

O que você deve ver

pnpm test: três arquivos — 3 passed em packages/db (o teste 3 agora também confere a 0002), 5 passed e 5 passed em apps/api (ambiente e rotas). Total 13.

3. Aplique a migração de segurança:

```
pnpm db:migrar
```

O que você deve ver

1 migracao(oes) aplicada(s). — migracoes no banco: 3 de 3

Se disser “Nada novo para aplicar”, a 0002 já estava lá. Se falhar com “permission denied for schema auth” ou parecido, a DATABASE_URL não é a do Supabase (seção 6 do Módulo 3).

4. Confira:

```
pnpm verificar
```

O que você deve ver no fim

[7/7] Seguranca por linha - RLS (Modulo 4)

ok RLS ligado nas 5 tabelas

ok 14 politicas no schema public (esperadas: 14)

ok funcao papel_atual() existe

ok auditoria: usuarios nao podem alterar nem apagar

Tudo certo. Proximo passo: pnpm dev (cinco luzes verdes em http://localhost:3000) e pnpm api:testar-fluxo

5.1 Ver no painel do Supabase (clique a clique)

1. **Table Editor** > clique em **perfis**. No topo da tabela, ao lado do nome, aparece a etiqueta **RLS enabled** (antes era um aviso vermelho “RLS disabled”). Confira nas cinco.

2. Menu lateral > **Authentication** > **Policies**. Cada tabela lista suas políticas com os nomes da migração (“perfis: ver o proprio”, ...). Clique numa para ver o SQL — é o mesmo do arquivo. **Não edite aqui**: mudança de política é migração (ADR-0004).

3. **Database** > **Functions**: papel_atual aparece com **Security: definir**.

6. Provar de ponta a ponta

Aqui não tem SQL Editor. O roteiro faz o que o site fará no Módulo 5, só que pelo terminal, e limpa tudo no fim.

1. No terminal do VS Code, suba os servidores:

```
pnpm dev
```

2. Abra um **segundo** terminal: no VS Code, botão + no canto do painel do terminal, ou **Terminal > New Terminal**, ou Ctrl+Shift+`. Nele:

```
pnpm api:testar-fluxo
```

O que você deve ver

```
ok API respondeu em http://localhost:3333
ok usuario fluxo-1724...@exemplo.com criado (uuid)
ok login ok, token recebido
ok GET /perfis/eu antes do cadastro: 404 (esperado)
ok POST /perfis com dados errados: 400 com detalhes
ok POST /perfis: 201, perfil criado com auditoria
ok POST /perfis de novo: 409 (ja existe)
ok GET /perfis/eu: 200 com papel paciente
ok GET /medicos: 200 (0 medico(s))
ok RLS: o usuario ve exatamente 1 perfil (o dele)
ok RLS: tentar virar admin foi recusado
ok RLS: apagar auditoria foi recusado (42501)
ok RLS: usuario comum nao le a auditoria (0 linhas)

Tudo certo: cadastro pela API com auditoria, e RLS protegendo o banco.
ok usuario de teste apagado (...)

Se algum passo falhar, a última linha diz qual ("Falhou no passo: 3 - rotas da API") e o usuário de teste é apagado
mesmo assim. Sobrou um fluxo-...@exemplo.com em Authentication > Users? Apague à mão; foi uma execução
interrompida.
```

Se aparecer...	Causa	Faça
a API nao respondeu em http://localhost:3333	O pnpm dev não está rodando no outro terminal	Suba e rode de novo.
nao criou o usuario: ...	SUPABASE_URL ou SUPABASE_SECRET_KEY erradas, ou a chave é a publishable	pnpm verificar; a luz Supabase do painel também estará vermelha.
NEXT_PUBLIC_SUPABASE_PUBLIC_KEY nao encontrada	O roteiro lê a chave pública de apps/web/.env.local para fazer login como o site	Refaça a seção 6 do Módulo 2.
login falhou: Email not confirmed	Confirmação de e-mail obrigatória ligada no projeto e o roteiro não conseguiu confirmar	Authentication > Providers > Email > desligue "Confirm email" no telemedicina-dev (só em dev).
esperava 201, veio 400: LOGIN_NAO_ENCONTRADO	A API está apontando para um banco diferente do projeto onde o usuário foi criado	SUPABASE_URL e DATABASE_URL têm de ser do mesmo projeto.

6.1 O que acabou de ser provado

- A API só atende com token válido (401 sem ele) e recusa dados errados com a lista exata do que consertar (400).
- O cadastro é atômico: perfil, paciente e a linha de auditoria entram juntos (201), e repetir dá 409.
- Fora da API, com a chave pública e um login válido, o usuário vê só o próprio perfil, não consegue se promover, não lê nem apaga a auditoria — o RLS está entre ele e o prontuário.

- Apagar o login apaga o perfil (cascata da 0001); a auditoria fica, com quem = null — o rastro sobrevive à pessoa.

Veja a auditoria como admin não dá ainda (não existe admin). No SQL Editor do painel (que usa o dono do banco, sem RLS): `select quando, acao, tabela, detalhes, ip from auditoria order by id desc limit 5;` — a linha `perfil.criado` do teste está lá, com o IP de onde veio.

7. As cinco luzes

Com o `pnpm dev` ainda rodando, abra `http://localhost:3000`. A quinta luz, **Seguranca por linha (RLS)**, diz “RLS ligado nas 5 tabelas, 14 politicas”. E `http://localhost:3333/saude` ganhou o bloco:

"seguranca":{"estado":"protegido","tabelasSemRls":[],"politicas":14}		
Luz vermelha em RLS, com o texto...	Causa	Faça
(todas as luzes) Sem resposta em <code>http://localhost:3333</code>	A API não subiu. No terminal, <code>apps/api dev:</code> mostra “Cannot find package '@supabase/supabase-js’” (faltou o <code>pnpm install</code>) ou “EADDRINUSE :::3333” (API antiga ainda viva)	<code>Ctrl+C</code> , <code>pnpm install</code> , <code>pnpm dev</code> . Para a porta presa: <code>Get-NetTCPConnection -LocalPort 3333 % { Stop-Process -Id \$_.OwningProcess -Force }</code>
RLS DESLIGADO em: ...	A 0002 não foi aplicada, ou alguém desligou o RLS no painel	<code>pnpm db:migrar</code> . Se já rodou: <code>Table Editor > tabela > ... > Enable RLS</code> , e descubra quem desligou.
RLS ligado, mas faltam politicas ou permissoes	Política apagada no painel, ou o REVOKE da auditoria desfeito	<code>pnpm db:verificar-rls</code> diz qual. Refaça pelo SQL da 0002.
Depende da luz do banco	O banco não respondeu	Resolva a luz Banco primeiro (Módulo 3, seção 8).

8. O que cada arquivo novo faz

Arquivo	O que faz
packages/db/drizzle/0002_rls_e_politicas.sql	Manual (--custom). papel_atual() SECURITY DEFINER; ENABLE ROW LEVEL SECURITY nas 5; 14 CREATE POLICY comentadas; REVOKE UPDATE, DELETE ON auditoria FROM anon, authenticated.
packages/db/src/estado-banco.ts	inspecionarBanco() agora lê pg_class.relrowsecurity (RLS por tabela), pg_policies (contagem), role_table_grants (auditoria) e pg_proc (papel_atual). rlsCompleto() resume. POLITICAS_ESPERADAS = 14: se você adicionar política, atualize aqui e o teste 3 lembra.
packages/db/src/verificar-rls.ts	pnpm db:verificar-rls: quatro linhas ok/ERRO com o comando que resolve.
apps/api/src/autenticacao.ts	criarAutenticadorSupabase() (supabase.auth.getUser), extrairToken(), criarExigirLogin() (o preHandler) e o tipo Usuario em req.usuario.
apps/api/src/auditoria.ts	registrarAuditoria(tx, {quem, acao, tabela, registroId, detalhes?, ip?}). Aceita o banco ou uma transação.
apps/api/src/erros.ts	ErroHttp(status, codigo, mensagem, detalhes?) e tratarErro(): o setErrorHandler do Fastify.
apps/api/src/rotas/perfis.ts	Schema Zod de POST /perfis (UFs válidas, CRM 4-7 dígitos, CPF 11 dígitos, data AAAA-MM-DD, bloco obrigatório conforme o papel); buscarPerfil() com dois LEFT JOIN; as duas rotas; tradução dos erros 23505 (409) e 23503 (400).
apps/api/src/rotas/medicos.ts	GET /medicos com INNER JOIN em perfis, ordenado por nome.
apps/api/src/rotas.test.ts	5 testes com autenticador falso: 401 sem token, token inválido, sem "Bearer"; 400 com lista de campos; 400 para papel admin.
apps/api/src/servidor.ts	criarServidor({ambiente?, autenticar?}); setErrorHandler; registra as rotas; /saude com o bloco segurança; logger silencioso em NODE_ENV=test.
apps/api/src/testar-fluxo.ts	O roteiro da seção 6. Usa a chave secreta só para criar/apagar o usuário; tudo o mais é feito como o site fará.
packages/shared/src/index.ts	StatusSaude.seguranca; Papel; PerfilResposta; MedicoResumo — o contrato entre API e site.
docs/adr-0006-rls-como-ultima-defesa.md	A decisão deste módulo, com as seis regras.

9. Como será toda tabela nova daqui para a frente

1. Schema em TypeScript (Módulo 3, seção 9).
2. `pnpm db:gerar --name nome_da_tabela` gera a migração da tabela.
3. **Na mesma hora**, `pnpm db:gerar --custom --name rls_nome_da_tabela` e escreva: `ENABLE ROW LEVEL SECURITY` e as políticas. Some o número de políticas em `POLITICAS_ESPERADAS`.
4. Se a tabela guarda dado clínico: política de `SELECT` só para paciente dono, médico com consulta e admin; nada de `UPDATE/DELETE` para usuários; toda escrita pela API com auditoria.
5. `pnpm db:migrar`, `pnpm verificar`, `pnpm test`, `commit`.

Regra de ouro

Tabela sem RLS não existe neste projeto. O `pnpm verificar` e a quinta luz ficam vermelhos na hora — e é para ficar.

10. Git: commit do módulo


```
git status
git add .
git commit -m "feat(seguranca): RLS e politicas (0002), porteiro e rotas de perfil com auditoria"
git push
```

Como sempre: nenhum `.env` na lista do `git status`. O `pnpm verificar` também acusa.

11. Checklist do Módulo 4

- ☐ Zip extraído por cima; `pnpm install`; `pnpm typecheck` limpo; `pnpm test` com 13 passed
- ☐ `pnpm db:migrar` aplicou a 0002 (3 de 3)
- ☐ `pnpm verificar` termina em “Tudo certo” com o passo [7/7] todo ok
- ☐ Painel: as 5 tabelas com “RLS enabled”; Authentication > Policies lista as 14
- ☐ `pnpm api:testar-fluxo` terminou em “Tudo certo” e apagou o usuário de teste
- ☐ `pnpm dev`: cinco luzes verdes
- ☐ Você sabe explicar a diferença entre USING e WITH CHECK, e por que `paper_atual()` é SECURITY DEFINER
- ☐ Você sabe por que a chave secreta ignora o RLS e por que isso obriga a API a ser quem grava
- ☐ Commit feito e enviado, sem nenhum `.env`

Próximo: Módulo 5 — Login e as primeiras telas. O site ganha cadastro, login e logout com o Supabase Auth (a chave pública, agora protegida pelo RLS), a tela “meu perfil” chamando `GET /perfis/eu` e o formulário que chama `POST /perfis`. O painel de luzes vira uma página interna de diagnóstico. Mesmo padrão: cada tela com o que você deve ver, tudo testado antes de ser entregue.